Mini-DLSS: A Lightweight Temporal Video Super-Resolution System

Research-Paper-Style Technical Breakdown and Engineering Whitepaper

Repository: `/Users/dhruvashekdar/Documents/Mini-DLSS`

Generated on May 23, 2026

Scope: fixed $2\times$ video super-resolution, temporal reconstruction, reproducible training/evaluation, and ONNX-oriented deployment

Abstract

Mini-DLSS is a compact research and engineering implementation of temporal video super-resolution (VSR). The repository is explicitly DLSS-inspired, but it is not an implementation of NVIDIA DLSS: it does not integrate with a renderer, use motion vectors from a game engine, or reproduce proprietary reconstruction logic. Instead, it isolates a public-data, PyTorch-based VSR problem: given a short low-resolution temporal window, reconstruct the high-resolution center frame at a fixed scale factor of $2\times$. This formulation makes the repository useful as a controlled study of temporal information, recurrent propagation, evaluation protocols, and deployment mechanics for learned video upscaling.

The engineering problem is the gap between single-image super-resolution and stable video reconstruction. A framewise super-resolution model can improve individual frames, but it cannot explicitly use information in neighboring frames and can produce temporal flicker. A video model must recover spatial detail while maintaining temporal coherence, which creates a coupled systems problem: the dataset must preserve sequence order, the model must ingest temporally aligned windows, evaluation must measure both per-frame fidelity and frame-to-frame stability, and inference must produce a runtime artifact such as an MP4 or ONNX model. Mini-DLSS addresses that problem by combining manifest-driven sequence datasets, a single-frame residual CNN baseline, a lightweight BasicVSR-style temporal model, reproducible training scripts, image and temporal metrics, qualitative media outputs, and a simple deployment path.

The core learned model, `BasicVSRMini`, is a deliberately simplified derivative of the BasicVSR design philosophy [1]. It extracts per-frame convolutional features, propagates context forward and backward through convolutional GRU cells, fuses the center feature with both recurrent states, and reconstructs the center high-resolution frame through a PixelShuffle upsampler [2]. Unlike full BasicVSR or EDVR-style methods [1], [3], the implementation does not estimate optical flow for alignment and does not emit a full high-resolution sequence in one forward pass. This is an important engineering tradeoff: it reduces implementation complexity and makes Colab-scale experiments feasible, but it limits the motion-compensation capability of the model and increases redundant computation during sliding-window inference.

The repository's pipeline is intentionally explicit. `MultiFrameSRDataset` reads sequence manifests, loads RGB frames as normalized tensors, constructs odd-length temporal windows, optionally creates low-resolution inputs through bicubic downsampling, and returns a center-frame target. `train.py` builds PyTorch data loaders, constructs models through a factory, optimizes an L_1 or L_1-plus-perceptual objective with Adam [4], validates at fixed intervals, and writes self-contained checkpoints. `eval.py` evaluates bicubic and learned models with cropped Y-channel and RGB PSNR/SSIM [5], computes temporal stability using global frame-difference energy and optional Farnebäck-flow temporal PSNR [6], and writes tables, JSON, image strips, and comparison videos. `demo_video.py` runs model or bicubic inference on arbitrary low-resolution MP4 input, while `export_onnx.py` exports a fixed-window model with dynamic batch and spatial axes for ONNX-oriented deployment.

The current codebase should be read as a reproducible VSR research scaffold rather than a finished state-of-the-art model. Its strengths are clear module boundaries, deterministic split/config handling, concrete baselines, reproducible artifacts, and a mathematically inspectable temporal model. Its main limitations are the absence of learned or explicit alignment, no conventional automated test suite, no cached recurrent-state runtime path, global rather than low-texture-masked temporal difference energy, and local Week 3 results that are explicitly marked as Vimeo-derived REDS-style validation rather than official REDS benchmark claims. These limitations are not incidental; they define the next research steps: official benchmark evaluation, true temporal/warping losses, flow- or deformable-alignment modules, broader ablations, runtime caching, quantization or mixed precision, and stricter experiment validation.

Index Terms

Video super-resolution, temporal reconstruction, BasicVSR, ConvGRU, PixelShuffle, PSNR, SSIM, ONNX, reproducible machine learning systems.

I. Introduction

Video super-resolution asks for high-resolution video reconstruction from low-resolution observations. In the simplest supervised formulation, a model observes low-resolution frames $\{\mathbf{x}_t\}$ and predicts high-resolution frames $\{\mathbf{y}_t\}$. Compared with single-image super-resolution, VSR can exploit sub-pixel motion, repeated texture observations, and temporal redundancy. It is also more difficult: adjacent frames may contain occlusions, motion blur, rolling shutter artifacts, inconsistent degradation, and disoccluded regions. A model that optimizes each frame independently may score reasonably on per-frame metrics but still shimmer during playback.

Mini-DLSS narrows this problem to a fixed and inspectable setting. The scale factor is locked to $s = 2$; the default temporal model uses $T = 5$ low-resolution input frames; the output is the high-resolution center frame; training uses Vimeo-90K-style sequence manifests [7]; and evaluation is designed around REDS-style validation and demo clips [8]. The project is small enough that the entire system can be understood from a few top-level entry points, yet complete enough to cover the practical lifecycle of a learned restoration model: dataset preparation, model training, validation, artifact generation, benchmarking, visualization, export, and runtime demo.

The motivating engineering problem is not merely "make images larger." Bicubic interpolation already solves geometric resizing. The harder problem is reconstructing plausible high-frequency detail while avoiding temporal instability and preserving a reproducible evaluation contract. This repository therefore treats the VSR system as a pipeline rather than as a model alone. It includes split manifests, dataset checks, training configs, baselines, metric code, tables, videos, checkpoints, and deployment scripts. The result is an internal research scaffold that a new engineer can run, inspect, and extend.

Existing approaches form a spectrum. Bicubic upsampling is deterministic, fast, and stable, but cannot synthesize missing detail beyond interpolation. Single-image SR models such as residual CNNs can learn sharper framewise mappings, but do not explicitly use neighboring frames. Modern VSR systems such as BasicVSR, BasicVSR++, and EDVR use temporal propagation and alignment to aggregate information across frames [1], [3], [9]. Mini-DLSS adopts the recurrent bidirectional-propagation idea, but strips it down to a ConvGRU-based center-frame predictor without explicit optical-flow alignment. That makes the codebase suitable for studying the theory-to-implementation relationship without hiding the core behavior behind a large framework.

II. Repository Objective

A. Purpose

The repository objective is to build a Mini-DLSS-style VSR pipeline for fixed $2\times$ upscaling. The project README states the task, baseline set, model path, evaluation protocol, and definition of done. The intended workflow is:

1) prepare or verify sequence datasets and split manifests;
2) train a single-frame or temporal super-resolution model;
3) evaluate against bicubic and learned baselines using image and temporal metrics;
4) generate qualitative comparison media;
5) export or run the trained model on an MP4 clip.

The name "Mini-DLSS" should be interpreted as an engineering analogy, not a claim of equivalence. DLSS-like systems use temporal reconstruction to upscale rendered content, but this repository operates on public VSR datasets and ordinary video files. It does not use engine-side motion vectors, depth buffers, exposure history, jitter patterns, or proprietary temporal antialiasing mechanisms.

B. Target Use Case

The target user is an ML/software engineer experimenting with lightweight video restoration. The repo supports:

- fast smoke and local validation cycles;
- Colab-oriented training through small configs and a notebook;
- comparison of bicubic, framewise SR, and temporal SR;
- reproducible result artifacts under `results/`;
- ONNX export for model portability;
- MP4 demo inference for visually inspecting temporal behavior.

C. Non-Goals

The implementation is not a game-engine plugin, not a production video codec, and not a state-of-the-art VSR benchmark submission. It intentionally omits several high-performance components: optical-flow alignment in the model, deformable convolution, sequence-wide recurrent-state caching during demo inference, distributed training, automatic mixed precision, quantization, and a full test suite.

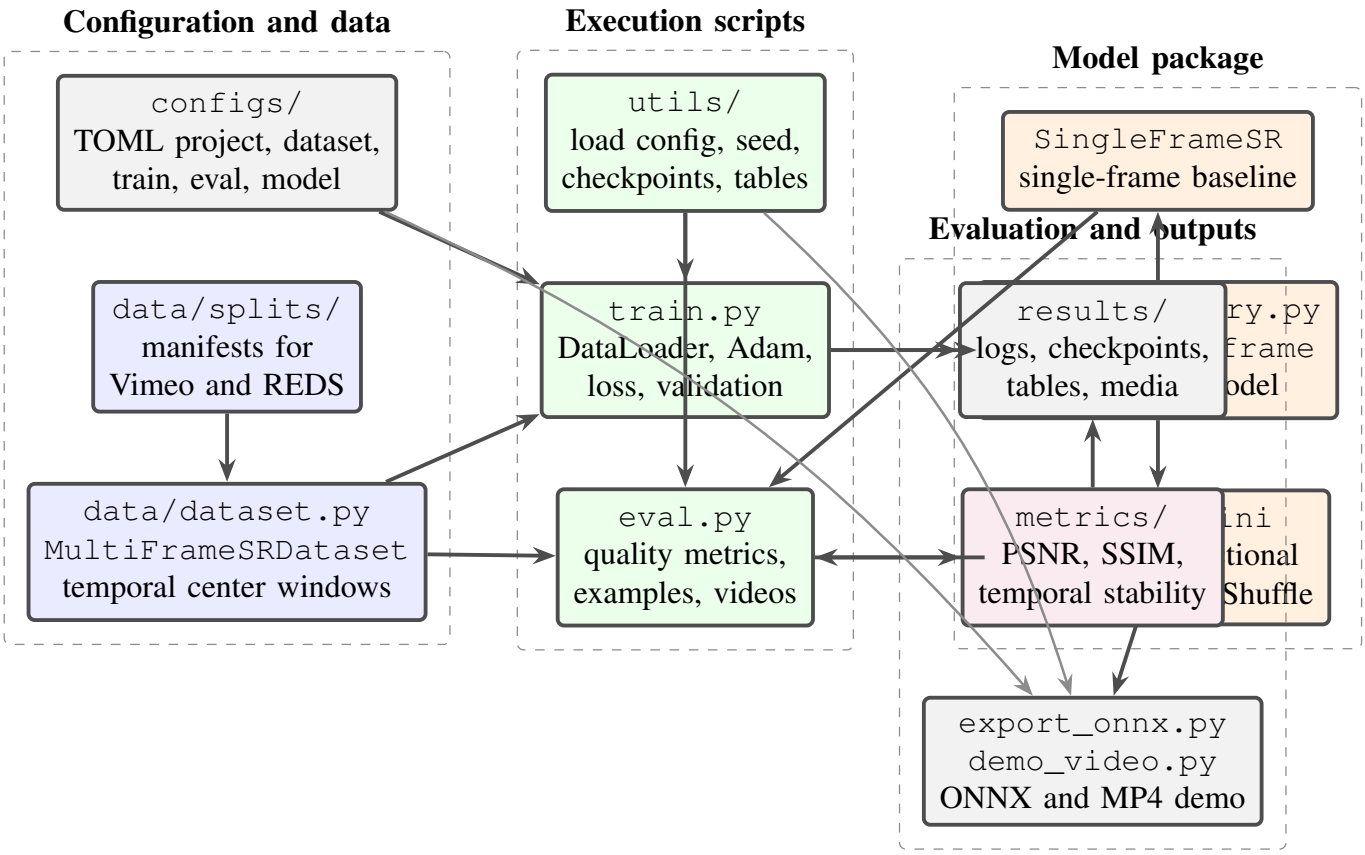

Fig. 1. Mini-DLSS repository architecture. Config files drive data loading, model construction, training, evaluation, and deployment scripts while shared utilities handle reproducibility and artifacts.

III. System Architecture

Mini-DLSS is organized around a narrow set of contracts, summarized in Figure 1. Configs specify dataset paths, manifests, scale, training schedule, and model hyperparameters. `MultiFrameSRDataset` converts a sequence manifest into tensor samples. The model package exposes a single factory function, `build_model`, which isolates entry points from architecture-specific constructors. Metrics accept tensors or NumPy arrays and do not depend on training internals. Utilities handle configuration loading, deterministic seeding, checkpoint save/load, and Markdown table writing. Top-level scripts compose these reusable units into training, evaluation, export, and demo workflows.

This architecture is simple, but it is not accidental. The project avoids a monolithic experiment framework so that the execution path remains readable. The top-level scripts are imperative and close to the PyTorch primitives they use. The cost is less abstraction for large experiment management; the benefit is that a new contributor can trace the code path from command line arguments to model outputs in one sitting.

Table I expands the architecture into concrete repository paths and interfaces.

IV. Codebase Breakdown

A. Configuration Layer

The configuration layer is intentionally flat. `utils/config.py` loads TOML or JSON and applies recursive JSON overrides through `deep_update`. This gives the command-line scripts a stable base configuration while allowing path substitutions on local machines or Colab. `seed_everything` fixes Python, NumPy, PyTorch, and cuDNN deterministic settings. This improves reproducibility but may reduce raw cuDNN performance because `benchmark=False`.

The default scale is $s = 2$ across configs. `temporal_small.toml` uses a 5-frame temporal window, 48 base channels, 6 residual blocks, batch size 4, 50k max steps, and L_1 loss. `temporal_full.toml` uses 64

TABLE I
MAJOR REPOSITORY MODULES AND RESPONSIBILITIES.

Path	Responsibility	Important interfaces and artifacts
`README.md`	Defines project scope, datasets, baselines, evaluation protocol, compute assumptions, and command checklist.	Documents fixed $2\times$ scale, Vimeo-90K training, REDS-style evaluation, PSNR/SSIM/tPSNR reporting, ONNX export, and demo flow.
`configs/`	Stores reproducible experiment definitions.	`temporal_small.toml`, `temporal_full.toml`, `single_frame_sr.toml`, and `base_2x.toml`.
`data/`	Implements dataset loading, split manifests, and dataset preparation scripts.	`MultiFrameSRDataset`, `read_manifest`, split freezing, union-root symlinking, dataset checks, and local REDS-style generation.
`models/`	Contains neural architectures and factory construction.	`SingleFrameSR`, `BasicVSRMini`, `ResidualBlock`, `ConvGRUCell`, `build_model`.
`metrics/`	Implements image and temporal evaluation.	Y/RGB PSNR, Y/RGB SSIM, border crop, frame-difference energy, optional Farnebäck-flow tPSNR.
`utils/`	Provides shared infrastructure.	TOML/JSON config loading, nested overrides, deterministic seeding, checkpoint save/load, Markdown table formatting.
`train.py`	Main optimization loop.	Data loaders, model construction, L_1/perceptual loss, Adam, validation, checkpointing, JSONL logs.
`eval.py`	Main validation and reporting CLI.	Bicubic/model/auto modes, metric aggregation, JSON/Markdown tables, PNG strips, MP4 comparison videos.
`demo_video.py`	Runtime MP4 inference path.	Reads LR MP4 frames, builds edge-padded temporal windows, writes SR MP4, reports ms/frame.
`export_onnx.py`	Deployment export path.	Exports configured checkpoint with dynamic batch and spatial axes but fixed temporal length.
`scripts/`	Shell orchestration.	Fast cycle, full cycle, and temporal-window ablation scripts.
`results/`	Experiment outputs and current evidence.	Checkpoints, logs, JSON/Markdown metrics, plots, image samples, and comparison videos.

base channels, 10 residual blocks, batch size 2, 300k max steps, a larger crop, and L_1 plus perceptual loss. `single_frame_sr.toml` uses one frame, 64 channels, 8 blocks, and batch size 8.

B. Data Layer

The central data abstraction is `MultiFrameSRDataset`. A manifest line is interpreted as a sequence ID relative to `hr_root` and optionally `lr_root`. The loader lists image files per sequence, verifies matching HR/LR frame counts when LR files exist, and builds center-frame sample indices. For odd $T = 2r + 1$, valid centers are $r, \ldots, N - r - 1$, so a sequence with N frames contributes $N - T + 1$ samples.

Each item returns:

$$\texttt{lr} \in [T, 3, h, w], \qquad \texttt{hr} \in [3, H, W], \tag{1}$$

plus `sequence_id` and `frame_name`. If stored LR frames are absent and `generate_lr_on_the_fly=True`, LR frames are generated by bicubic downsampling of HR frames. Training optionally applies an HR crop aligned

to the scale factor, then maps the same crop to LR coordinates. Random horizontal, vertical, and temporal-order flips provide lightweight augmentation.

The data scripts support practical dataset hygiene. `freeze_vimeo_splits.py` creates deterministic Vimeo train/val/test manifests from official list files. `freeze_vimeo_union_splits.py` deduplicates two Kaggle Vimeo subsets and creates a deterministic union split. `build_vimeo_union_root.py` symlinks sequence directories into a merged root. `create_reds_val_from_vimeo.py` creates a local REDS-style validation tree from Vimeo frames for smoke testing when official REDS is unavailable. `check_dataset.py` verifies sequence counts, temporal window counts, LR/HR alignment, and sample tensor shapes.

C. Model Layer

The model layer is compact. `models/common.py` defines `ResidualBlock` and `ConvGRUCell`. `models/single_f` defines `SingleFrameSR`. `models/temporal_basicvsr.py` defines `BasicVSRMini`. `models/factory.py` maps config names to constructors. This package has no dependency on datasets, metrics, OpenCV, or result artifacts.

The single-frame baseline accepts either a 4D tensor $[B, 3, h, w]$ or a 5D temporal tensor $[B, T, 3, h, w]$. In the temporal case it selects the center frame. The temporal model requires a 5D input and predicts only the center output frame. This shared tensor convention lets `train.py`, `eval.py`, and `export_onnx.py` build either model through the same factory.

D. Metrics Layer

`metrics/image_metrics.py` provides PSNR and SSIM on cropped tensors. Evaluation crops `scale` pixels from each border before computing metrics, then reports primary Y-channel metrics and optional RGB metrics. The Y transform uses a BT.601-style studio-range formula. `metrics/temporal_metrics.py` reports global frame-difference energy for all sequences and, when OpenCV is available, computes Farnebäck optical flow and reports tPSNR after warping adjacent outputs.

One important implementation/documentation mismatch is visible here. The top-level project README describes the fallback temporal metric as frame-to-frame difference energy in low-texture regions, but the implemented `frame_difference_energy` computes a global mean absolute difference without a low-texture mask. This does not break the code, but it matters for interpreting temporal stability claims.

E. Workflow Entry Points

`train.py` is the main learning loop. It parses a config and optional JSON override, selects CPU if CUDA is requested but unavailable, seeds random number generators, builds train/validation data loaders, constructs the model, creates Adam, optionally restores a checkpoint, then iterates until `max_steps`. It logs training loss to `train_log.jsonl`, validates at configured intervals, saves `best.pt` when PSNR-Y improves, saves `latest.pt`, writes `val_metrics.jsonl`, and optionally writes step checkpoints.

`eval.py` supports `bicubic`, `model`, and `auto` modes. In auto/model mode it attempts to load `results/runs/<run_r` unless an explicit checkpoint is provided. If no checkpoint exists in either auto or model mode, the current implementation warns and falls back to bicubic evaluation. It writes Markdown and JSON metrics, sample PNG strips, and curated MP4 comparisons.

`demo_video.py` is the direct runtime demonstration path. It reads a low-resolution MP4 with OpenCV, converts frames to tensors, builds an edge-replicated temporal window for each output frame, runs bicubic or model inference, writes an output MP4, and reports elapsed seconds plus milliseconds per frame. Unlike `eval.py`, model-mode demo inference raises an error if the checkpoint is missing. `export_onnx.py` loads a required checkpoint and exports the configured model with dynamic batch, height, and width axes; temporal length remains fixed by `dataset.num_frames`.

V. Training, Evaluation, and Inference Pipeline

A. Input Processing

Figure 2 shows the training-time data path from config and manifests through temporal sampling, model optimization, validation, and checkpoint artifacts.

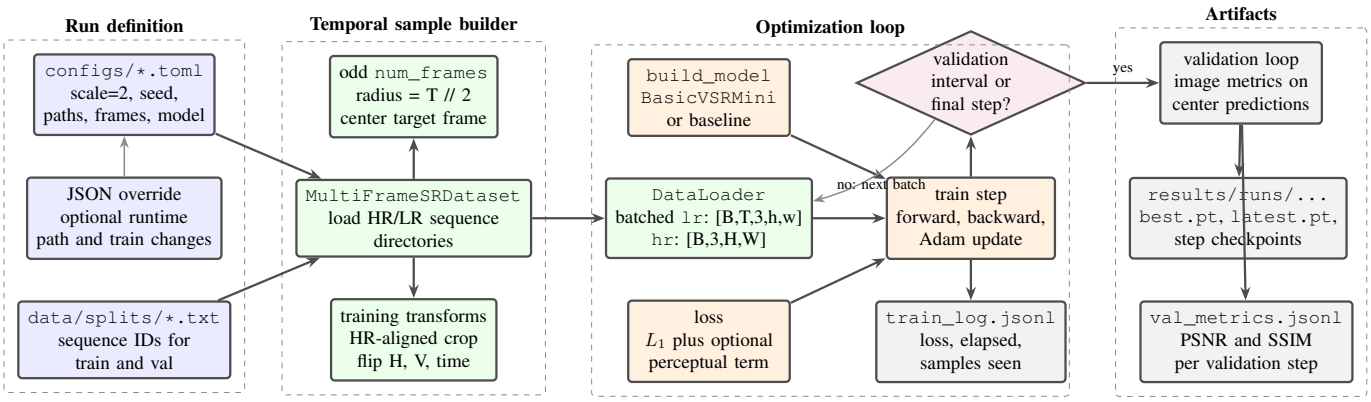

Fig. 2. Training pipeline implemented by `train.py`. Configs and manifests instantiate temporal windows, the model predicts the center HR frame, and validation controls checkpoint and metric artifacts.

The data pipeline starts from sequence directories and text manifests. A manifest does not list individual frames; it lists sequence IDs. This is important because temporal samples must be formed from neighboring frames in sorted order. For a sequence ID q, the dataset expects:

$$\texttt{hr\_root}/q/\{f_0, \ldots, f_{N-1}\}, \tag{2}$$

and optionally:

$$\texttt{lr\_root}/q/\{f_0, \ldots, f_{N-1}\}. \tag{3}$$

If `lr_root` is missing, the dataset can synthesize LR frames by bicubic downsampling. When LR frames are provided, the dataset verifies frame counts but does not explicitly verify spatial scale alignment; spatial mismatches would surface later through tensor shape or metric behavior. The split files currently include 12,824 union Vimeo training sequences, 674 union Vimeo validation sequences, 1,748 union Vimeo test sequences, 240 REDS train IDs, 20 REDS validation IDs, and 10 REDS test IDs.

B. Feature Generation

The repository does not precompute learned features. Feature generation occurs inside the model forward pass. For each LR window, `BasicVSRMini` slices the temporal tensor into T frames, runs a shared feature extractor on each frame, and stores a Python list of feature maps. This design is simple and ONNX-exportable for fixed T, but it does not exploit recurrent-state reuse across overlapping runtime windows.

C. Training Flow

The training objective is supervised center-frame restoration:

$$\hat{\mathbf{y}}_c = f_\theta(\mathbf{x}_{c-r}, \ldots, \mathbf{x}_c, \ldots, \mathbf{x}_{c+r}), \qquad T = 2r + 1. \tag{4}$$

Each optimization step pulls a batch from an infinite dataloader cycle, sends LR and HR tensors to the selected device, computes the prediction, computes the configured loss, backpropagates, and applies an Adam update. Validation is run at `val_interval` or at the final step. The validation loop clamps model outputs to $[0, 1]$ before metrics; the training loss uses the raw model output.

D. Evaluation Flow

Evaluation uses batch size 1 and preserves sequence order for temporal metrics. For each sample, `eval.py` computes a bicubic center-frame reference, a nearest-neighbor upsampled LR visualization panel, and either a model prediction or bicubic prediction depending on mode. Per-frame image metrics are accumulated into sample-weighted global averages. Predictions are also bucketed by `sequence_id`; after per-sample evaluation, each bucket is sorted by frame name and passed to `compute_temporal_metrics`, and temporal scores are averaged equally across sequences. This two-stage design keeps per-frame and per-sequence metrics separate.

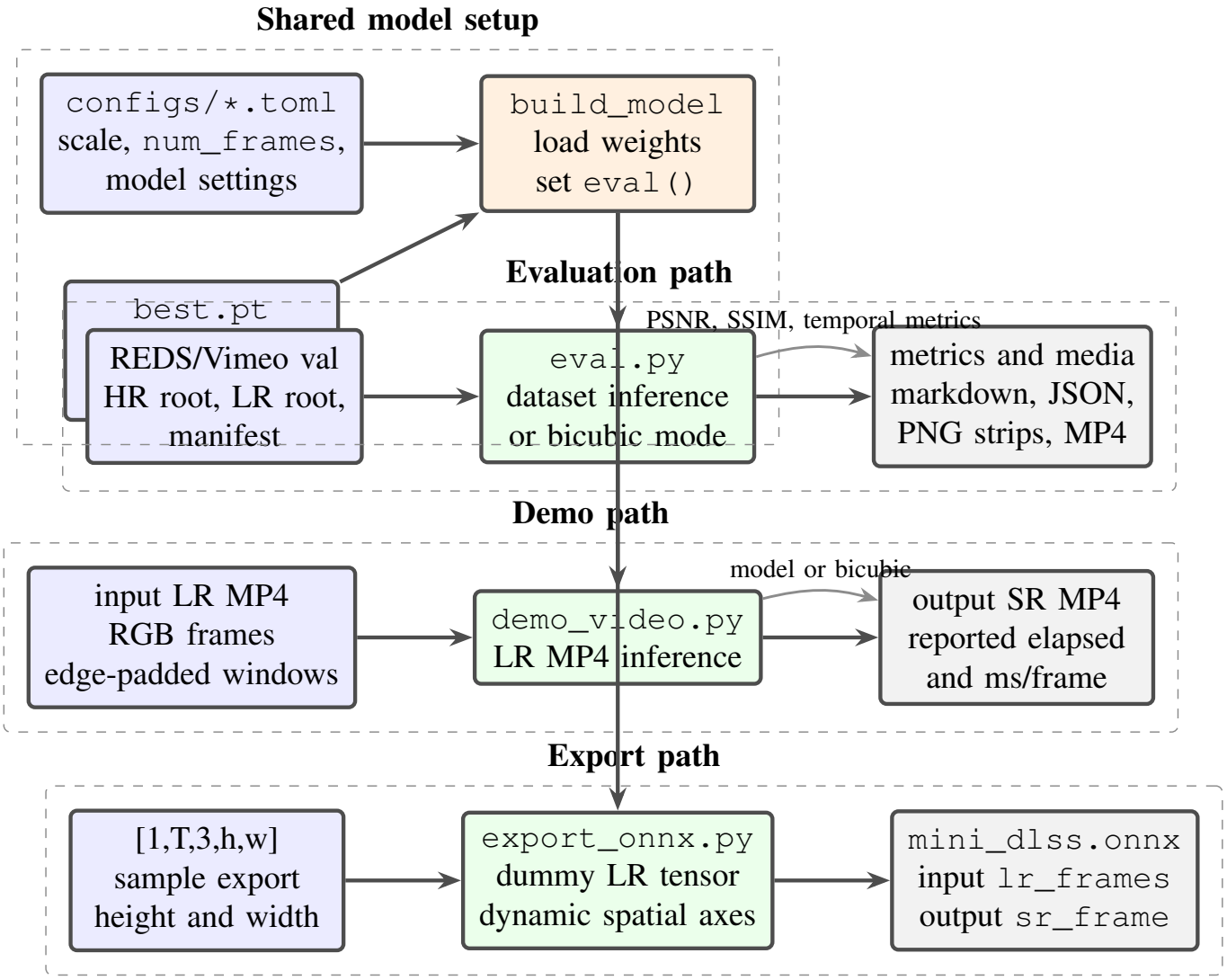

Fig. 3. Inference and deployment paths. The same config-driven model setup feeds validation evaluation, MP4 demo generation, and ONNX export.

E. Inference and Deployment Flow

Runtime MP4 inference follows the three deployment paths in Figure 3. The MP4 path is sliding-window center-frame prediction. For output frame i, `demo_video.py` builds indices:

$$\mathcal{I}(i) = \{\max(0, \min(n-1, j)) \mid j = i-r, \ldots, i+r\}, \tag{5}$$

so boundary frames are replicated. The model sees one window and emits one SR frame. This strategy is robust for arbitrary-length video, but recomputes overlapping feature extraction and recurrent states. ONNX export uses a dummy tensor of shape $[1, T, 3, h, w]$ and declares dynamic axes for batch and spatial dimensions only. The fixed temporal axis is consistent with model construction from config and simplifies downstream runtime assumptions.

F. Logging, Checkpointing, and Metrics

Training writes two JSONL streams: `train_log.jsonl` for step, loss, elapsed time, and samples seen; and `val_metrics.jsonl` for validation PSNR/SSIM metrics by step. Checkpoints store model state, optimizer state, step, best metric, and the full config. This makes `best.pt` and `latest.pt` sufficient to resume or audit a run. Evaluation writes Markdown and JSON metrics under `results/tables/` and qualitative media under `results/videos/`.

VI. Mathematical Foundations

A. Observation and Degradation Model

Let $\mathbf{y}_t \in [0,1]^{3\times H\times W}$ denote the high-resolution RGB frame at time t, and let $\mathbf{x}_t \in [0,1]^{3\times h\times w}$ denote the low-resolution observation. For the fixed scale factor $s = 2$, the aligned ideal dimensions are $H = sh$ and $W = sw$. When LR frames are generated on the fly, the implemented degradation is bicubic interpolation:

$$\mathbf{x}_t = \mathcal{D}_s(\mathbf{y}_t) = \mathrm{Bicubic}(\mathbf{y}_t; \lfloor H/s \rfloor, \lfloor W/s \rfloor), \tag{6}$$

using PyTorch interpolation with `align_corners=False`. Bicubic interpolation can be understood as separable cubic convolution [10]: each output sample is a weighted combination of nearby input samples along horizontal and vertical axes. The implementation delegates the exact kernel behavior to PyTorch.

The supervised learning problem is center-frame reconstruction from an odd temporal window:

$$\hat{\mathbf{y}}_c = f_\theta(\mathbf{x}_{c-r:c+r}), \quad T = 2r + 1. \tag{7}$$

The dataset restricts c so all temporal neighbors exist during dataset evaluation. Runtime MP4 inference instead pads the boundary by replication.

B. Residual Convolutional Blocks

Both learned models use residual blocks:

$$\mathcal{R}(\mathbf{z}) = \mathbf{z} + W_2 * \sigma(W_1 * \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2, \tag{8}$$

where $*$ is a 2D convolution and σ is ReLU. Residual connections improve optimization by allowing the block to learn a correction relative to the identity mapping, a principle widely used in image restoration architectures such as EDSR [11].

The single-frame baseline maps the center LR frame through:

$$\mathbf{f}_0 = H(\mathbf{x}_c), \tag{9}$$

$$\mathbf{f} = \mathbf{f}_0 + B(\mathbf{f}_0), \tag{10}$$

$$\hat{\mathbf{y}}_c = U(\mathbf{f}), \tag{11}$$

where H is the convolutional head, B is a stack of residual blocks, and U is the upsampling tail. The implementation has no bicubic residual skip; the network directly predicts RGB values.

C. BasicVSRMini Temporal Propagation

Figure 4 diagrams the tensor flow through `BasicVSRMini`. For a temporal window $\{\mathbf{x}_{c-r}, \ldots, \mathbf{x}_{c+r}\}$, the model extracts per-frame features:

$$\mathbf{e}_t = E(\mathbf{x}_t), \qquad \mathbf{e}_t \in \mathbb{R}^{C\times h\times w}. \tag{12}$$

It then runs two convolutional GRUs, one forward and one backward:

$$\mathbf{h}_t^{\rightarrow} = G_f(\mathbf{e}_t, \mathbf{h}_{t-1}^{\rightarrow}), \tag{13}$$

$$\mathbf{h}_t^{\leftarrow} = G_b(\mathbf{e}_t, \mathbf{h}_{t+1}^{\leftarrow}). \tag{14}$$

The cell itself follows the GRU gating structure [12], with dense affine maps replaced by convolutions as shown in Eq. (17). To avoid overloading the raw-frame notation, let $\mathbf{u}_t$ denote the feature input to a ConvGRU cell:

$$[\mathbf{z}_t, \mathbf{r}_t] = \sigma\left(W_{zr} * [\mathbf{u}_t, \mathbf{h}_{t-1}] + \mathbf{b}_{zr}\right), \tag{15}$$

$$\tilde{\mathbf{h}}_t = \tanh\left(W_h * [\mathbf{u}_t, \mathbf{r}_t \odot \mathbf{h}_{t-1}] + \mathbf{b}_h\right), \tag{16}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{17}$$

The update gate $\mathbf{z}_t$ controls how much new candidate state replaces old state; the reset gate $\mathbf{r}_t$ controls how much previous state participates in candidate formation. Because all gates are convolutional, spatial layout is preserved.

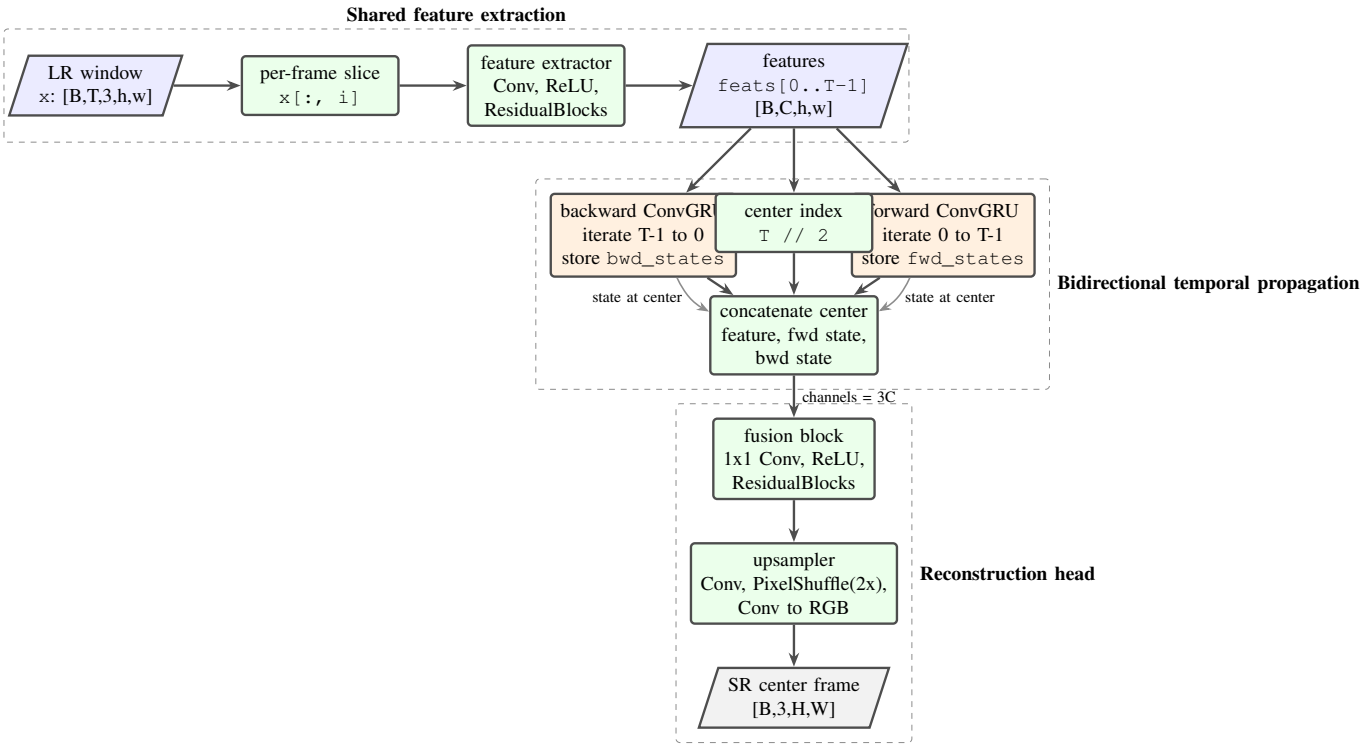

Fig. 4. `BasicVSRMini` model flow. Each LR temporal window is encoded frame by frame, propagated in both directions with ConvGRU cells, fused at the center frame, and upsampled with PixelShuffle.

At the center index c, the model concatenates the center feature and both propagated states:

$$\mathbf{g}_c = F\left([\mathbf{e}_c, \mathbf{h}_c^{\rightarrow}, \mathbf{h}_c^{\leftarrow}]\right), \tag{18}$$

then reconstructs:

$$\hat{\mathbf{y}}_c = U(\mathbf{g}_c). \tag{19}$$

This matches the propagation-and-aggregation intuition of BasicVSR [1], but not its full alignment machinery. There is no optical-flow warping before recurrence and no deformable alignment. Motion handling is therefore implicit in convolutional state dynamics.

D. PixelShuffle Reconstruction

The upsampling tail uses sub-pixel convolution [2]. A convolution first produces Cs^2 channels at LR spatial resolution, then applies the rearrangement in Eq. (21):

$$\mathbf{q} \in \mathbb{R}^{B\times Cs^2\times h\times w}. \tag{20}$$

PixelShuffle rearranges channel groups into spatial positions:

$$\mathrm{PS}_s(\mathbf{q})_{b,c,sp+i,sr+j} = \mathbf{q}_{b,cs^2+is+j,p,r}, \quad 0 \le i,j < s. \tag{21}$$

The final convolution maps the upsampled feature tensor to three RGB channels.

E. Loss Functions

The base objective is mean absolute reconstruction error, given in Eq. (22):

$$\mathcal{L}_1(\theta) = \frac{1}{|\Omega|}\sum_{u\in\Omega} \left| f_\theta(\mathbf{x}_{c-r:c+r})_u - \mathbf{y}_{c,u} \right|, \tag{22}$$

where Ω indexes all pixels and channels in a batch. L_1 is common in restoration because it is less sensitive to outliers than L_2 and often produces sharper reconstructions.

The full temporal config optionally adds the VGG16 feature loss in Eq. (23) [13], [14]:

$$\mathcal{L}(\theta) = \mathcal{L}_1(\theta) + \lambda \left\| \Phi(f_\theta(\mathbf{x}_{c-r:c+r})) - \Phi(\mathbf{y}_c) \right\|_1 . \tag{23}$$

`PerceptualLoss` uses torchvision VGG16 features through layer index 15, freezes their weights, and evaluates them in inference mode. If torchvision weights are unavailable, the implementation returns zero perceptual loss and prints a warning. This fallback improves portability but weakens strict reproducibility: a config requesting perceptual loss may fall back to L_1-only at runtime, and the warning can be missed in long logs.

F. Optimization

Training uses Adam [4]. For gradient $\mathbf{g}_t = \nabla_\theta \mathcal{L}_t$, Adam maintains exponential moving averages:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \tag{24}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \tag{25}$$

applies bias correction:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \qquad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}, \tag{26}$$

and updates:

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \tag{27}$$

The configs use learning rate 2×10^{-4} and rely on PyTorch defaults for the remaining Adam hyperparameters.

G. Image Quality Metrics

For evaluation, predictions and targets are first clamped to $[0, 1]$ and cropped by s pixels on each border. The Y-channel conversion is:

$$Y = \frac{65.481R + 128.553G + 24.966B + 16}{255}. \tag{28}$$

PSNR is computed with normalized peak value 1, as in Eq. (29):

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2, \qquad \mathrm{PSNR} = -10\log_{10}(\mathrm{MSE}). \tag{29}$$

The implementation caps near-perfect MSE cases at 100 dB.

SSIM compares local luminance, contrast, and structure [5]. With Gaussian-window means μ_x, μ_y, variances σ_x^2, σ_y^2, covariance σ_{xy}, and constants $C_1 = 0.01^2$, $C_2 = 0.03^2$, the implemented expression is Eq. (30):

$$\mathrm{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}. \tag{30}$$

The code uses an 11×11 Gaussian window with $\sigma = 1.5$ and grouped convolution over channels.

H. Temporal Stability Metrics

The always-available temporal metric is global frame-difference energy:

$$E_\Delta = \frac{1}{M-1}\sum_{t=2}^{M}\frac{1}{CHW}\left\|\hat{\mathbf{y}}_t - \hat{\mathbf{y}}_{t-1}\right\|_1 . \tag{31}$$

Lower values indicate smaller average adjacent-frame changes, but this metric is not motion-compensated and may penalize legitimate motion.

When OpenCV [15] is available, the repository also computes Farnebäck dense optical flow [6] between consecutive predicted frames and warps the current frame toward the previous frame:

$$\mathrm{tPSNR} = \frac{1}{M-1}\sum_{t=2}^{M}\mathrm{PSNR}\left(\mathrm{warp}(\hat{\mathbf{y}}_t, \mathbf{F}_{t-1\to t}), \hat{\mathbf{y}}_{t-1}\right). \tag{32}$$

TABLE II
CONFIGURED MODEL SIZES AND APPROXIMATE COMPUTE SCALE. MAC ESTIMATES USE LR CROP DIMENSIONS IMPLIED BY THE HR CROP AND $s = 2$.

Config	C	Blocks	Parameters	Approx. training-crop MACs
`single_frame_sr.toml`	64	8	0.742M	3.06G at LR 64×64
`temporal_small.toml`	48	6	0.716M	9.08G at LR 64×64
`temporal_full.toml`	64	10	1.714M	31.8G at LR 80×80

The implementation constructs remap coordinates from the previous-frame grid plus the estimated flow, so the formula should be read as an implementation-defined forward-flow sampling convention rather than as a general inverse-warp derivation. This is a stronger temporal stability signal than raw difference energy because it partially accounts for motion, but it inherits the failure modes of optical flow, especially around occlusions, low texture, reflections, and large displacement.

I. Computational Complexity

Let $A = hw$ denote LR spatial area, C base channels, s scale, N reconstruction residual blocks, T temporal frames, and $K = \max(1, \lfloor N/2 \rfloor)$ feature-extractor residual blocks in `BasicVSRMini`. Counting convolution multiply-accumulates and ignoring activation/PixelShuffle overhead, the single-frame baseline has approximately:

$$P_{\text{SISR}} = (18N + 9s^2)C^2 + (55 + 2N + s^2)C + 3 \tag{33}$$

parameters and:

$$M_{\text{SISR}} = A[(18N + 9s^2)C^2 + (27 + 27s^2)C] \tag{34}$$

MACs.

The temporal model has approximately:

$$P_{\text{VSR}} = (18K + 108 + 3 + 18N + 9s^2)C^2 + (62 + 2K + 2N + s^2)C + 3, \tag{35}$$

where the $108C^2$ term comes from the two ConvGRU cells, and:

$$M_{\text{VSR}} = A[T(18KC^2 + 27C) + 108TC^2 + 3C^2 + 18NC^2 + 9s^2C^2 + 27s^2C]. \tag{36}$$

Table II connects the symbolic complexity to the actual configs inspected in the repository.

VII. RESEARCH CONTEXT AND DERIVATION

A. From Interpolation to Learned Super-Resolution

Bicubic interpolation is the baseline because it is deterministic, fast, and widely used in SR evaluation. It provides a useful lower-bound engineering reference: no training, negligible parameters, stable behavior, and low latency. Learned SR replaces fixed interpolation weights with data-driven nonlinear mappings. The single-frame baseline follows the residual CNN tradition of image restoration [11]; its mathematical role is to estimate $p(\mathbf{y}_c \mid \mathbf{x}_c)$ or, more practically, a point estimate minimizing expected reconstruction loss.

B. From Single-Frame SR to Temporal SR

Video contains redundant information across time. If an object edge appears at slightly different sub-pixel positions across adjacent frames, a temporal model can integrate those observations to reconstruct detail that is ambiguous in any single frame. The ideal temporal estimator conditions on a window:

$$f_\theta^\star = \arg\min_f \mathbb{E}\left[\ell(f(\mathbf{x}_{c-r:c+r}), \mathbf{y}_c)\right]. \tag{37}$$

The challenge is alignment. Full VSR systems often estimate motion or use deformable alignment before aggregation [3], [9]. Mini-DLSS instead uses recurrent convolutional state to propagate information. This makes alignment implicit and local: the model can learn motion-tolerant filters, but has no explicit warping operator inside the network.

TABLE III
MAJOR DESIGN TRADEOFFS IN THE CURRENT IMPLEMENTATION.

Decision	Benefit	Cost or risk
Fixed $2\times$ scale	Simplifies configs, crops, metric border handling, and ONNX export.	Does not exercise arbitrary-scale SR or model-conditioned scale factors.
Manifest-defined sequences	Reproducible split policy and scene-level control.	Requires directory layout discipline; bad manifests fail at dataset construction.
On-the-fly bicubic LR	Reduces stored LR data and makes smoke runs easy.	Adds CPU/data-loader work and may differ from official degradation.
Center-frame prediction	Simple target contract and metric handling.	Runtime windows overlap heavily; no sequence-wide output reuse.
ConvGRU temporal propagation	Lightweight temporal context without explicit flow modules.	Limited motion alignment; hidden states remain LR-grid local.
PixelShuffle upsampling	Efficient learned upsampling at LR compute cost.	Can produce checkerboard-like artifacts if training is poor.
Default L_1 loss	Stable and easy to optimize.	May prefer averaged textures and does not directly enforce temporal consistency.
Optional perceptual loss	Encourages feature-space similarity in full runs.	VGG availability fallback can reduce the objective to L_1-only if the warning is missed.
Direct Python entry points	Easy to trace and modify.	Less scalable experiment management than Hydra/Lightning-style frameworks.
Global diff-energy temporal metric	Always available and simple.	Confounds true motion with flicker; the top-level project README's low-texture fallback is not implemented.
Farnebäck tPSNR	Motion-compensated temporal signal without learned flow.	Depends on OpenCV and flow quality; omitted if `cv2` is unavailable.
Deterministic cuDNN settings	Improves repeatability.	May reduce throughput on some GPUs.

C. Relationship to BasicVSR

BasicVSR emphasizes propagation, alignment, aggregation, and upsampling [1]. `BasicVSRMini` preserves propagation and aggregation in a simplified form:

- propagation: forward and backward ConvGRU recurrences;
- aggregation: concatenation of center feature, forward state, and backward state;
- upsampling: convolution plus PixelShuffle;
- omitted: optical-flow alignment, sequence-wide bidirectional output, and more advanced refinement.

Thus, the repo is best described as BasicVSR-style rather than BasicVSR-complete. This distinction is important for research claims and for explaining why quality may lag stronger VSR baselines.

D. Datasets and Protocol

Vimeo-90K was introduced with task-oriented flow for video enhancement tasks [7]; REDS was introduced in the NTIRE 2019 restoration context [8]. Mini-DLSS uses Vimeo-style data for training and REDS-style validation/demo sequences for reporting. The repo's local Week 3 results explicitly state that their REDS-style validation set is generated from Vimeo for pipeline progression, not official REDS. This is a good engineering practice: it separates pipeline verification from benchmark claims.

VIII. ENGINEERING TRADEOFFS

The tradeoffs in Table III are not hypothetical; each follows from a concrete code path or config choice in the current repository.

A. Scalability and Runtime

The temporal model is still small in parameter count, but compute grows linearly with temporal length T and spatial area A. The largest config is only 1.714M parameters, yet its estimated crop compute is much higher than the single-frame baseline because it extracts features for five frames and runs two recurrent passes. Inference latency in Table IV reflects this: the temporal model reports roughly 34.66 ms/frame versus 9.105 ms/frame for the single-frame fast-cycle model and 0.238 ms/frame for bicubic in that local setup.

B. Memory

During training, memory is dominated by stored activations for T feature extractor passes, ConvGRU states, fusion residual blocks, and the upsampled output. Crop size is therefore a major lever. The full config uses batch size 2 and HR crop 160, while the small config uses batch size 4 and HR crop 128. This is a classic VSR tradeoff: larger crops improve spatial context but quickly increase activation memory.

C. Numerical Stability

Image tensors are normalized to $[0, 1]$. Evaluation clamps predictions before metrics, which avoids invalid PSNR/SSIM values from out-of-range outputs. Training does not clamp before loss, preserving gradients outside the display range. PSNR guards near-zero MSE by returning 100 dB. SSIM uses constants C_1 and C_2 from the standard formulation, reducing division instability in low-variance windows.

D. Parallelization and GPU Considerations

The implementation uses PyTorch [16], data loaders, and ordinary GPU tensor execution. However, per-frame feature extraction is expressed as a Python list comprehension across T, and recurrent passes are Python loops. For $T = 5$ this is acceptable; for larger temporal windows, batching the frame dimension through the feature extractor and improving export/runtime structure would matter. The data loader uses multiple workers and pinned memory, but on-the-fly bicubic downsampling may become a CPU bottleneck.

E. Failure Cases

Likely failure modes include large motion, occlusion, repeated textures, thin high-frequency structures, scene cuts inside temporal windows, incorrect LR/HR alignment, and mismatched degradation between train and validation data. The current model has no explicit scene-cut handling and no alignment module, so temporal windows containing inconsistent content can degrade the center prediction. The evaluation pipeline also assumes sorted filenames reflect temporal order.

IX. End-to-End Example

A typical fast-cycle run begins with split and dataset verification, then training:

```
python train.py --config configs/temporal_small.toml \
  --override '{"dataset":{"train_hr_root":"/path/to/vimeo/hr",
  "val_hr_root":"/path/to/reds/hr","val_lr_root":"/path/to/reds/lr"}}'
```

At startup, `train.py` loads the TOML config, applies the JSON override, seeds the process, builds `MultiFrameSRDataset` for train and validation, wraps them in data loaders, constructs `BasicVSRMini`, creates Adam, and starts the step loop. Every validation interval, it computes cropped PSNR/SSIM and updates `best.pt` when PSNR-Y improves.

Evaluation can then be run explicitly:

```
python eval.py --config configs/temporal_small.toml \
  --checkpoint results/runs/temporal_vsr_5f_small_2x/checkpoints/best.pt
```

`eval.py` reads the validation manifest, builds one-sample batches, computes predictions, accumulates metrics, buckets outputs by sequence, computes temporal metrics, writes Markdown/JSON results, saves PNG image strips, and optionally creates MP4 comparisons for IDs in `data/splits/reds_demo_clips.txt`. Each strip contains four panels for the evaluated mode: nearest-neighbor LR-upsampled center frame, bicubic reference, current

TABLE IV
EXISTING WEEK 3 LOCAL REDS-STYLE COMPARISON. THESE NUMBERS ARE USEFUL PIPELINE EVIDENCE, BUT THE REPO MARKS THIS VALIDATION SET AS VIMEO-DERIVED RATHER THAN OFFICIAL REDS.

Method	Params	PSNR-Y	SSIM-Y	tPSNR	Diff. energy	ms/frame
Bicubic	0.000M	36.8033	0.9601	36.3441	0.0217	0.238
Single-frame SR fast cycle	0.742M	32.3755	0.8807	33.8817	0.0183	9.105
Temporal SR fast cycle, 600 steps	0.716M	29.2989	0.8532	37.6005	0.0134	34.660
Temporal SR extended local, 2000 steps	0.716M	33.5138	0.9247	37.6004	0.0175	34.660

prediction, and HR target. It does not automatically combine bicubic, single-frame, and temporal predictions into the same strip unless those outputs are assembled separately.

For runtime demonstration:

```
python demo_video.py --input input_lr.mp4 --output output_sr.mp4 \
  --config configs/temporal_full.toml \
  --checkpoint results/runs/temporal_vsr_5f_full_2x/checkpoints/best.pt
```

The script reads the MP4, builds edge-padded windows, predicts one SR frame per LR frame, writes the output video, and reports milliseconds per frame. For export:

```
python export_onnx.py --config configs/temporal_full.toml \
  --checkpoint results/runs/temporal_vsr_5f_full_2x/checkpoints/best.pt \
  --output results/onnx/mini_dlss_temporal_full.onnx
```

The model is exported with input name `lr_frames` and output name `sr_frame` using ONNX [17].

A. Results Snapshot

Table IV illustrates a useful engineering lesson. Early learned models can underperform bicubic in PSNR while improving or changing temporal metrics. The extended temporal run improves substantially over the fast temporal run but still trails bicubic on this local validation setup. This does not invalidate the pipeline; it shows why the repository's definition of done requires disciplined benchmark runs and why future work should include stronger training, official REDS validation, and ablations.

X. THEORY-TO-IMPLEMENTATION MAPPING

Table V is the strongest evidence that the repository is internally coherent. The mathematical formulation is not merely described in prose; every object has a concrete implementation counterpart. Where the implementation diverges from idealized VSR theory, the divergence is visible: alignment is implicit rather than flow-based, temporal loss is not used during training, and runtime windows are recomputed rather than streamed.

XI. CONCLUSION

Mini-DLSS is a well-scoped educational and research engineering scaffold for temporal video super-resolution. Its strongest qualities are explicit data contracts, small and readable model definitions, reproducible configs, clear baselines, deterministic split handling, checkpointed training, metric reporting, qualitative artifacts, and a direct MP4/ONNX deployment path. It is technically deeper than a minimal SR demo because it includes temporal windows, bidirectional recurrent propagation, Y-channel evaluation, temporal metrics, and artifact discipline.

The main weaknesses are also clear. The temporal model is BasicVSR-style but not BasicVSR-complete; it lacks explicit motion alignment, temporal training loss, sequence-wide output, and recurrent-state caching. The local results are pipeline evidence rather than official benchmark evidence. The fallback temporal metric described in the top-level project README is not exactly the one implemented. The perceptual loss fallback improves portability but can blur experiment semantics. Finally, the repository would benefit from automated tests around dataset indexing, metric formulas, checkpoint resume, ONNX export, and demo boundary windows.

TABLE V
MAPPING FROM THEORETICAL OBJECTS TO IMPLEMENTATION ARTIFACTS.

Theory object	Implementation	Notes
HR frame $\mathbf{y}_t$	`hr_window[self.radius]` in `MultiFrameSRDataset`	Center frame target only.
LR observation $\mathbf{x}_t$	Loaded LR frame or bicubic downsampled HR frame	Generated via `F.interpolate(..., mode="bicubic")`.
Temporal window $\mathbf{x}_{c-r:c+r}$	`lr` tensor with shape $[T, 3, h, w]$	Batched to $[B, T, 3, h, w]$.
Estimator f_θ	`SingleFrameSR` or `BasicVSRMini`	Built by `build_model`.
Residual operator $\mathcal{R}$	`ResidualBlock`	Two 3×3 convs plus identity skip.
Bidirectional recurrence	`forward_gru`, `backward_gru`	ConvGRU states on LR grid.
Upsampling operator U	Conv + `PixelShuffle(scale)` + Conv	Learned RGB reconstruction.
Objective $\mathcal{L}$	L_1, optional VGG perceptual term	Perceptual term depends on torchvision/VGG availability.
Optimizer	`torch.optim.Adam`	Learning rate from config.
Image fidelity	`compute_image_metrics`	Border crop, Y-channel and RGB PSNR/SSIM.
Temporal stability	`compute_temporal_metrics`	Global diff energy plus optional Farnebäck tPSNR.
Deployment graph	`torch.onnx.export`	Dynamic batch/spatial axes; fixed temporal window.

Future improvements should prioritize official REDS evaluation, ablations over temporal length and loss design, explicit temporal consistency losses, flow- or deformable-alignment modules, mixed precision, runtime feature/state caching, ONNX Runtime benchmarking, and stricter experiment validation. With those additions, Mini-DLSS could evolve from a compact VSR scaffold into a stronger comparative research platform.

REFERENCES

[1] K. C. K. Chan, X. Wang, K. Yu, C. Dong, and C. C. Loy, "BasicVSR: The search for essential components in video super-resolution and beyond," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 4947–4956.

[2] W. Shi, J. Caballero, F. Huszar, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 1874–1883.

[3] X. Wang, K. C. K. Chan, K. Yu, C. Dong, and C. C. Loy, "EDVR: Video restoration with enhanced deformable convolutional networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2019.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *International Conference on Learning Representations*, 2015.

[5] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[6] G. Farnebäck, "Two-frame motion estimation based on polynomial expansion," in *Image Analysis*, ser. Lecture Notes in Computer Science, vol. 2749. Springer, 2003, pp. 363–370.

[7] T. Xue, B. Chen, J. Wu, D. Wei, and W. T. Freeman, "Video enhancement with task-oriented flow," *International Journal of Computer Vision*, vol. 127, no. 8, pp. 1106–1125, 2019.

[8] S. Nah, S. Baik, S. Hong, G. Moon, S. Son, R. Timofte, and K. M. Lee, "NTIRE 2019 challenge on video deblurring and super-resolution: Dataset and study," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2019.

[9] K. C. K. Chan, S. Zhou, X. Xu, and C. C. Loy, "BasicVSR++: Improving video super-resolution with enhanced propagation and alignment," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2022.

[10] R. G. Keys, "Cubic convolution interpolation for digital image processing," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 29, no. 6, pp. 1153–1160, 1981.

[11] B. Lim, S. Son, H. Kim, S. Nah, and K. Mu Lee, "Enhanced deep residual networks for single image super-resolution," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2017.

[12] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734.

[13] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *International Conference on Learning Representations*, 2015.
[14] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual losses for real-time style transfer and super-resolution,” in *European Conference on Computer Vision*, 2016, pp. 694–711.
[15] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
[16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
[17] ONNX Community, “ONNX: Open neural network exchange,” https://onnx.ai/, 2017, accessed 2026-05-23.